全安全等级 Frodo 硬件实现评估报告

低资源开销设计（第一部分）

版本：Ver. 1.4

审核人：陆家昊

批准人：刘冬生

编写人：王家琛、赵正鹏

版本记录

序号	版本号	生成时间	主 要 修 改 记 录	作者
1	V1.0	20241118	创建	王家琛、赵正鹏
2	V1.1	20241120	框架定义与优化	陆家昊
3	V1.2	20241205	完成报告	王家琛、赵正鹏
4	V1.3	20241210	修改架构图错误，补充整体资源开销和周期估算	王家琛、赵正鹏
5	V1.4	20250724	根据最新版实现方案更新报告	王家琛、赵正鹏
6				
7				

目录

1. 系统架构设计 5
1.1. 系统整体架构设计 5
1.2. 方案性能评估 6
2. 算法指令设计及存储规划 7
2.1. 指令设计 7
2.1.1. TRANS 指令 7
2.1.2. MATMUL 指令 7
2.1.3. MATRAN 指令 8
2.1.4. ABSORB 指令 8
2.1.5. PKH 指令 8
2.1.6. SQUEEZE 指令 9
2.1.7. SS 指令 9
2.1.8. GENS 指令 9
2.1.9. GENE 指令 10
2.1.10. AMATMUL 10
2.1.11. MATADD 10
2.1.12. MATADD 11
2.1.13. ABSORB_S 11
2.1.14. SQUEEZE_EX 11
2.1.15. ABSORB_EX 11
2.2. 算法流程分析与指令使用 12
2.2.1. Key Generation 12
2.2.2. Encapsulation 13
2.2.3. Decapulation 14
2.3. 矩阵运算流程设计 15
2.4. SRAM 存储规划 18
2.5. 矩阵短边存储策略 19
3. 子模块原理与结构设计 20
3.1. Hash 系数生成模块 20
3.1.1. 接口设计 20
3.1.2. 结构与功能设计 21
3.2. 高斯采样模块 23
3.2.1. 接口设计 23
3.2.2. 结构与功能设计 23

3.3. 矩阵运算 PE 阵列.....25
3.3.1. 接口设计.....25
3.3.2. 结构与功能设计.....26
3.4. Encode 模块.....27
3.4.1. 接口设计.....27
3.4.2. 结构与功能设计.....28
3.5. Decode 模块.....29
3.5.1. 接口设计.....29
3.5.2. 结构与功能设计.....29
3.6. 控制模块.....30
3.6.1. 接口设计.....30
3.6.2. 功能设计.....32

1. 系统架构设计

1.1. 系统整体架构设计

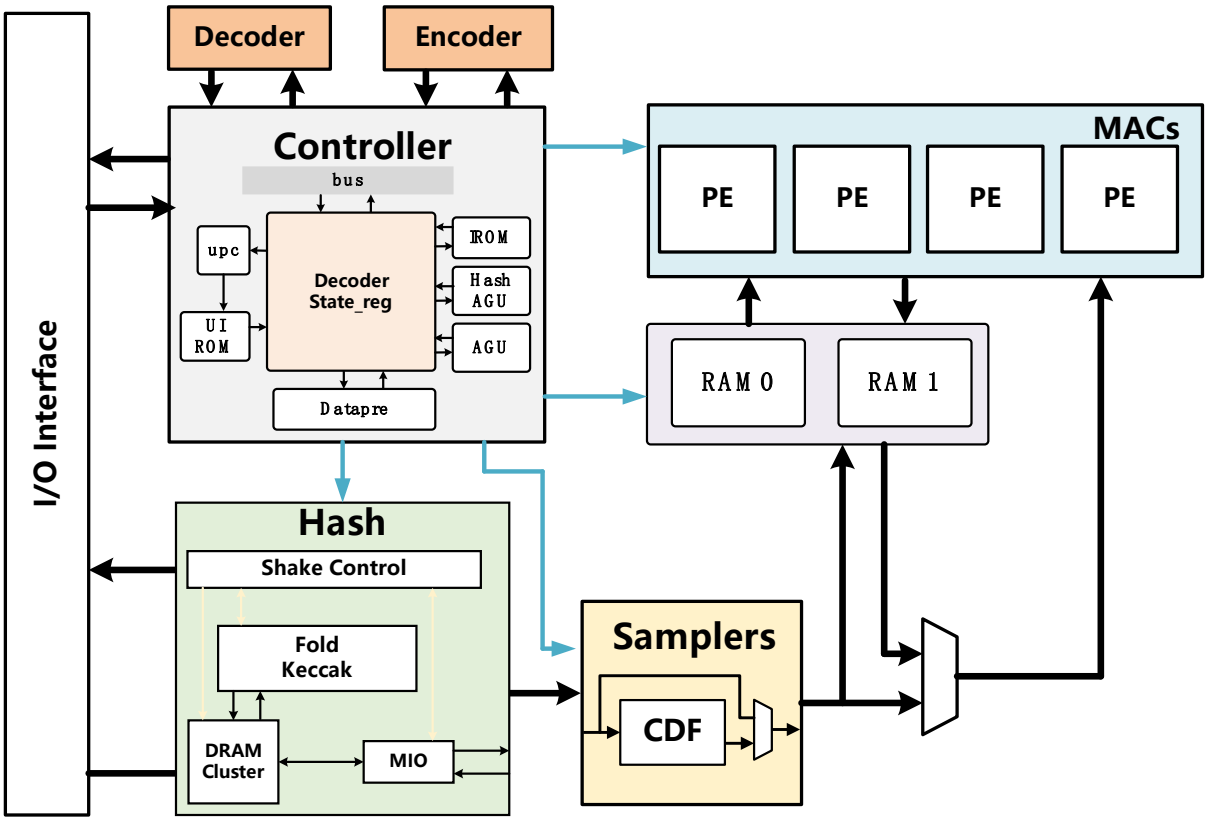


图 1 系统整体架构图

如图所示，低资源开销 Frodo 芯片主要包括**控制模块、SRAM 存储模块、PE 阵列、Hash 系数生成模块、采样模块、Encode 模块、Decode 模块与 I/O 接口**。各部分的功能描述如下：

控制模块：负责对指令进行解码，并控制计算状态转移，同时为 SRAM 存储模块、矩阵运算 PE 阵列、Hash 系数生成模块、采样模块提供相应的控制信号。

SRAM 存储模块：使用多块 SRAM 存储 Frodo 计算过程的中间值和结果、同时存储控制所需的指令地址索引与矩阵地址索引。

矩阵运算 PE 阵列：用于核心矩阵乘法加法操作。

Hash 系数生成模块：负责实现 SHA3 函数，用于生成摘要矩阵（如 A）、生成用于采样的摘要、生成数据摘要作为随机数种子（如 $seed_A$）、或生成摘要等待传输（如 pkh 或 ss）

采样模块：负责对 Hash 模块生成的随机数进行高斯采样，输出满足特定高斯分布特性的随机数，用于矩阵 S, E, S', E', E''的生成。

Encode 模块：负责将位串编码为矩阵元素进而组成矩阵，组成的矩阵用于参与矩阵

运算。

Decode 模块：负责将矩阵中的矩阵元素转换为位串，进而将整个矩阵转换为位串用于作为哈希系数生成模块的输入数据。

1.2. 方案性能评估

核心运算子模块的硬件结构以及整体方案已设计完成，下面给出各个核心子模块的硬件资源开销、总体方案所需资源开销评估、算法所需时钟周期数评估。

表 1 模块资源开销表

模块名称	资源 1	使用情况	资源 2	使用情况	资源 3	使用情况
Hash 系数生成模块	FF	14021	LUT	6383	BRAM	0
高斯采样模块	FF	126	LUT	32	BRAM	0
矩阵运算 PE 阵列	FF	1012	LUT	242	BRAM	0
SRAM 存储模块	FF	0	LUT	0	BRAM	16

表 2 总体所需资源开销评估

Resource	LUT	FF	BRAM
Utilization	29457	18356	16

表 3 各安全等级下算法所需时钟周期数评估

安全等级	Frodo—640	Frodo—976	Frodo—1344
KeyGen 时钟周期	2407362	4833007	8825839
Encaps 时钟周期	2433494	4869933	8875821
Decaps 时钟周期	2448696	4894820	8912228

2. 算法指令设计及存储规划

2.1. 指令设计

为降低控制开销，低资源开销的 Frodo 采用指令进行控制。本设计中指令为 27 位，具体的指令设计如下。

需要说明，指令中地址的格式分为两种：1，长度 4 位，其含义为地址索引，实际地址通过查表获得，即通过此地址读出存放在 RAM0 中的数据实际地址；2，长度 12 位，其含义为实际地址，配合 port(指示此地址用于 ram0 或 ram1)进行寻址。采用此种指令地址格式设计配合存储规划，以应对 FrodoKEM 运算过程中复杂的地址跳转。

2.1.1.TRANS 指令

110	0000(4bit)	Src(4bit)	Dst(4bit)	mode(1bit)	0…0(12bit)

图 2　TRANS 指令格式

说明：

- Src：待运算数据地址索引
- Dst：结果结果矩阵存储地址索引
- mode：运算模式
  - mode = 0：encode 运算
  - mode = 1：decode 运算

功能描述：

控制用于对矩阵进行 Decode 运算或对位串进行 Encode 运算。

2.1.2.MATMUL 指令

100	Src_s(4bit)	Src_l(4bit)	Dst(4bit)	signal(1bit)	0…0(12bit)

图 3　MATMUL 指令格式

说明：

- Src_s:：参与矩阵运算的乘法矩阵 1 地址索引
- Src_s:：参与矩阵运算的乘法矩阵 2 地址索引
- Dst：参与矩阵运算的加法矩阵地址索引
- signal：指示乘法是否取反

功能描述：

控制用于进行不涉及矩阵 A（哈希生成矩阵）的矩阵运算，可实现矩阵 1×矩阵 2＋加法矩阵的运算并将结果矩阵存储在加法矩阵位置。

2.1.3.MATRAN 指令

101	0000(4bit)	Src(4bit)	Dst(4bit)	mode(1bit)	0…0(12bit)

图 4　MATRAN 指令格式

说明：

- Src：待运算数据地址索引
- Dst：运算结果数据存储地址索引
- Mode：区分位宽

功能描述：

控制将 Src 处对应地址的矩阵转置后放入 Dst 处对应地址，mode 区分位宽

2.1.4.ABSORB 指令

000	Addr(12bit)	Length(8bit)	15to8(1bit)	port(1bit)	inverse(1bit)	00(2bit)	ram_reset(1bit)	0(1bit)

图 5　ABSORB 指令格式

说明：

- Addr：输入哈希模块数据的地址
- Length：输入哈希模块数据的长度（总位数/64）
- 15to8：指示是否将输入数据每 16 位取 15 位有效数据（专用于 Frodo640）
- port：指示地址用于的 ram
  - port = 0：用于 ram0
  - port = 1：用于 ram1
- inverse：指示从输入数据低位或高位开始装载入 Hash
  - inverse = 0，从低位开始装载
  - inverse = 1，从高位开始装载
- ram_reset：是否需要进行哈希内分布式 ram 的复位

功能描述：

控制输入数据进入哈希模块进行吸收

2.1.5.PKH 指令

000	Addr(12bit)	0…0(9bit)	port(1bit)	0…0(4bit)	1(1bit)

图 6　PKH 指令格式

说明：

- Addr：矩阵 B 地址
- port：指示地址用于的 ram

功能描述：

专用于 pkh 生成过程前哈希模块对矩阵 B 的吸收

2.1.6.SQUEEZE 指令

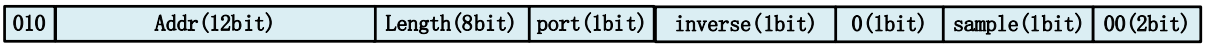


图 7　SQUEEZE 指令格式

说明：

- Addr：哈希模块输出数据的存储地址
- Length：哈希模块输出数据的长度（总位数/64）
- port：指示地址用于的 ram
  - port = 0：用于 ram0
  - port = 1：用于 ram1
- inverse：指示从输出数据低位或高位开始存入 ram
- sample：指示输出数据是否需要进行采样

功能描述：

控制哈希模块挤压输出数据，数据可直接存储或采样后进行存储

2.1.7.SS 指令



图 8　SS 指令格式

说明：

- Addr：生成 ss 过程中输入哈希模块矩阵 B 的地址
- flag：指示吸收 B 矩阵的部分
  - flag = 1：吸收上半部分
  - flag = 0：吸收下半部分

功能描述：

控制进行 ss 生成过程中矩阵 B 输入哈希与吸收的过程

2.1.8.GENS 指令

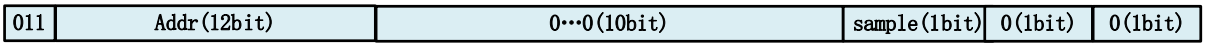


图 9　GENS 指令格式

说明：

- Addr：S 矩阵的存储地址
- sample：指示哈希输出数据是否需要进行采样

功能描述：

控制算法流程中 S 矩阵与 S’矩阵从哈希模块中的挤压生成与存储

2.1.9.GENE 指令

011	Addr(12bit)	0…0(7bit)	port(1bit)	0(1bit)	mode(1bit)	sample(1bit)	0(1bit)	1(1bit)

图 10　SQUEEZE 指令格式

说明：

- Addr：矩阵的存储地址
- port：指示地址用于的 ram
- mode：指示生成的矩阵
  - mode = 0：生成 E
  - mode = 1：生成 E’
- sample：指示输出数据是否需要进行采样

功能描述：

控制算法流程中 E 矩阵与 E’矩阵从哈希模块中的挤压生成与存储

2.1.10.AMATMUL

111	S(4bit)	$Seed_A$(4bit)	E(4bit)	0…0(13bit)

图 11　AMATUAL 指令格式

说明：

- S：乘法矩阵的地址索引
- $Seed_A$：$seed_A$ 的地址索引
- E：加法矩阵地址索引

功能描述：

控制算法流程中哈希矩阵 A 参与的矩阵运算

2.1.11.MATADD

100	0000(4bit)	Src_1(4bit)	Dst(4bit)	0…0(12bit)	1(1bit)

图 12　MATADD 指令格式

说明：

- Src_1：加法矩阵 1 的地址索引
- Dst：加法矩阵 2 的地址索引

功能描述：

控制矩阵加法运算，结果矩阵存储在加法矩阵 2 对应的地址中

2.1.12.MATADD

100	0000(4bit)	Src_l(4bit)	Dst(4bit)	0…0(12bit)	1(1bit)

图 13　MATADD 指令格式

说明：

- Src_l：加法矩阵 1 的地址索引
- Dst：加法矩阵 2 的地址索引

功能描述：

控制矩阵加法运算，结果矩阵存储在加法矩阵 2 对应的地址中

2.1.13.ABSORB_S

110	0…0(23bit)	ram_reset(1bit)	1(1bit)

图 14　ABSORB_S 指令格式

说明：

- ram_reset：是否需要哈希模块内部分布式 ram 复位

功能描述：

控制向哈希模块输入 0x5f、0x96 数据

2.1.14.SQUEEZE_EX

101	0…0(24bit)	1(1bit)

图 15　SQUEEZE_EX 指令格式

功能描述：

控制哈希模块执行一次挤压

2.1.15.ABSORB_EX

010	Addr(12bit)	Length(6bit)	0(1bit)	port(1bit)	0000(1bit)	1(1bit)

图 16　ABSORB_EX 指令格式

说明：

- Addr：输入数据地址
- Length：输入数据的长度（总位数/64）
- port：指示地址用于的 ram

功能描述：

控制向哈希模块执行一次数据输入吸收

2.2. 算法流程分析与指令使用

本设计通过对 FrodoKEM 算法流程的研究，设计了 FrodoKEM 硬件实现的总体算法流程。同时对原算法文档中的流程进行优化，有利于提高资源利用率，加快算法执行速度，下文以 FrodoKEM-1344 最高安全等级为例展示指令的使用。

2.2.1.Key Generation

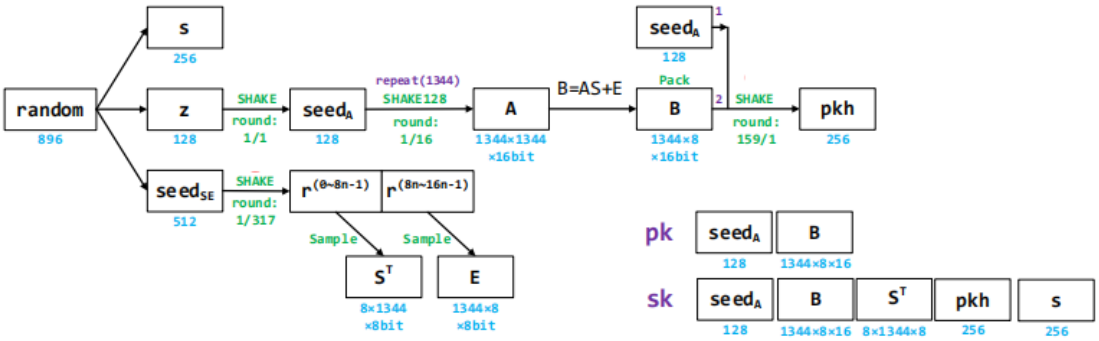


图 17　Keygen 流程

Keygen 流程指令实现如下：

```
11001011111000000000000000011 //吸收 5f     absorb_s
00011111110000000100001000000 //吸收 seedse  长度 512
01110101001000000000000100100 // genS
01100000001000000000000000101 //genE
00011111111000000001000000010 //吸收 z    absorb  长度 128
01000000000000000000100000001 //生成 seedA, absorb_ex,长度 128
01011111101000000001001000000 //生成 seedA,长度 128
11100010011001000000000000000 // B=AS+E
00011111101000000001001000010 //absorb  吸收 seedA,长度 128
00000000001000000000000000001 // pkh,吸收矩阵 B
01011111101001000010001000000 //  挤出 pkh,长度 256
```

2.2.2.Encapsulation

E

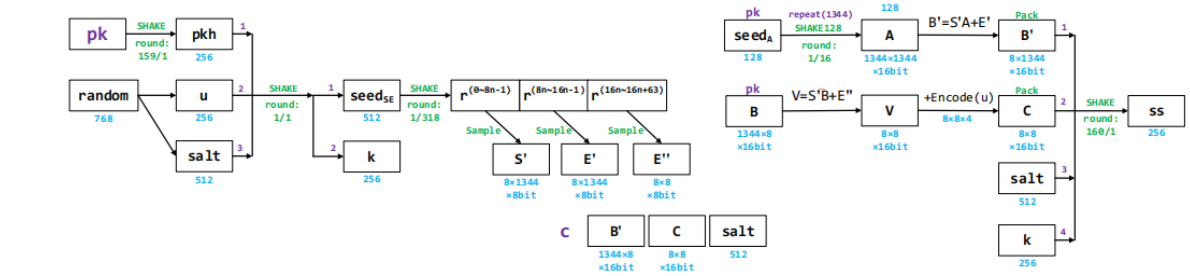


图 18　Enc 流程

Enc 流程指令实现如下：

```
00011111101000000000100100010 //absorb  吸收 seedA,长度 128
00000000001000000000000000001 // pkh,吸收矩阵 B
01011111101001000001000100000 //  挤出 pkh,长度 256
00011111101001000001000100010 //  吸收 pkh,长度 256
00011111111100100001000000000 //吸收 u,长度 256
00011111111110000010000100000 //吸收 salt，长度 512
01000000000000000100000000001 //执行吸收，长度 1024
01011111110000000010000100000 //生成 seedse，长度 512
01011111111011000001000000000 //生成 k，长度 256
11010010110000000000000000011 //吸收 96
00011111110000000010000100000 //吸收 seedse,长度 512
01110101001000000000000000100 //生成 S'
01100000001000000000000101101 //生成 E'
01011111101000000110100001l0//生成 E"前 13 行（832bit）
10100000000000000000000000001 //执行一次挤压
01011111101110100000110000100 //生成 E"后 3 行（640bit）
10101000100010100000000000000 //转置 E"
10001100010010100000000000000 //  生成 V_T
10101010101010000000000000000 //转置 V
11001110111010100000000000000 //encodeu
11101101001100000000000000001 // B'=S'A+E'
10001000100010100000000000001 //C = V+encode(u)
00100000001000000000000100000 //  吸收 B'前半段
00100000001000100000000100001 //吸收 B'后半段
00011111110100000111110110000 //吸收 C 前 960bit
```

```
01000000000000000100010000001 //执行一次吸收
00011111111101110000010110000 //吸收 C 后 64bit
00011111111110000010000100000 //吸收 salt 前 512bit
00011111111011000010000000000 //吸收 k 256bit
01000000000000000011010000001 //执行一次吸收,832
01011111111111000001000000000 //挤出 ss,256
```

2.2.3.Decapulation

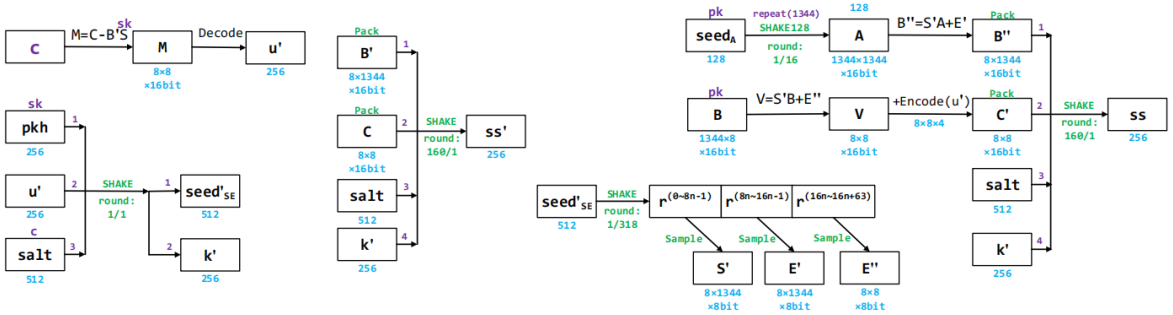


图 18　Dec 流程

Dec 流程指令实现如下：

```
10010111000010010000000000000 //M=C-B'S
11001000100101010000000000000 // u' = decode(u)
00011111101001000010001000010 //吸收 pkh,长度 256
00011111111001000010000000000 //吸收 u,长度 256
00011111111110000010000100000 //吸收 salt，长度 512
01000000000000000100000000001 //执行吸收，长度 1024
01011111110000000010000100000 //生成 seedse，长度 512
01011111111011000010000000000 //生成 k，长度 256
00100000001000000000000100000 // 吸收 B'前半段
00100000001000100000000100001 //吸收 B'后半段
00011111110100000111110110000 //吸收 C 前 960bit
01000000000000000100010000001 //执行一次吸收
00011111111101110000010110000 //吸收 C 后 64bit
00011111111110000010000100000 //吸收 salt 前 512bit
00011111111011000010000000000 //吸收 k 256bit
01000000000000000011010000001 //执行一次吸收,832
01011111101011000010000000000 //挤出 ss,256
11010010110000000000000000011 //吸收 96
```

```
00011111110000000100001000000 //吸收 seedse,长度 512
01110101001000000000000000100 //生成 S'
01100000001000000000000101101 //生成 E'
01011111101000000110100001l0//生成 E"前 13 行（832bit）
10100000000000000000000000001 //执行一次挤压
01011111101110100001100001000 //生成 E"后 3 行（640bit）
10101000100010100000000000000 //转置 E"
10001100010010100000000000000 // 生成 V_T
10101010101010000000000000000 //转置 V
11001110111010100000000000000 //encodeu
11101101001100000000000000001 // B'=S'A+E'
10001000100010100000000000001 //C = V+encode(u)
00100000001000000000000100000 // 吸收 B'前半段
00100000001000100000000100001 //吸收 B'后半段
00011111110100000111110110000 //吸收 C 前 960bit
01000000000000000100010000001 //执行一次吸收
00011111111011100000l0110000 //吸收 C 后 64bit
00011111111100000100001000000 //吸收 salt 前 512bit
00011111111011000010000000000 //吸收 k 256bit
01000000000000000011010000001 //执行一次吸收,832
01011111111110000010001000000 //挤出 ss,256
```

2.3. 矩阵运算流程设计

在本设计中，为节省存储空间，公共矩阵 A 不进行存储，而是在计算时进行实时生成，这给矩阵运算带来了难点，因为公共矩阵 A 必须由 Hash 模块按行生成，但是在封装和解封装的过程中，公共矩阵 A 都会作为右乘矩阵需要按列进行读取。受限于 Hash 模块的数量，难以使用传统的数据流进行矩阵计算，且需要额外的控制开销。考虑到在低资源开销的 Frodo 中，计算的并行度也有所降低，最终针对不同的矩阵乘法，设计了不同的数据流。下面针对三种矩阵运算类型，A 矩阵左乘、A 矩阵右乘，A 矩阵不参与运算，对矩阵运算流程设计进行详细说明。

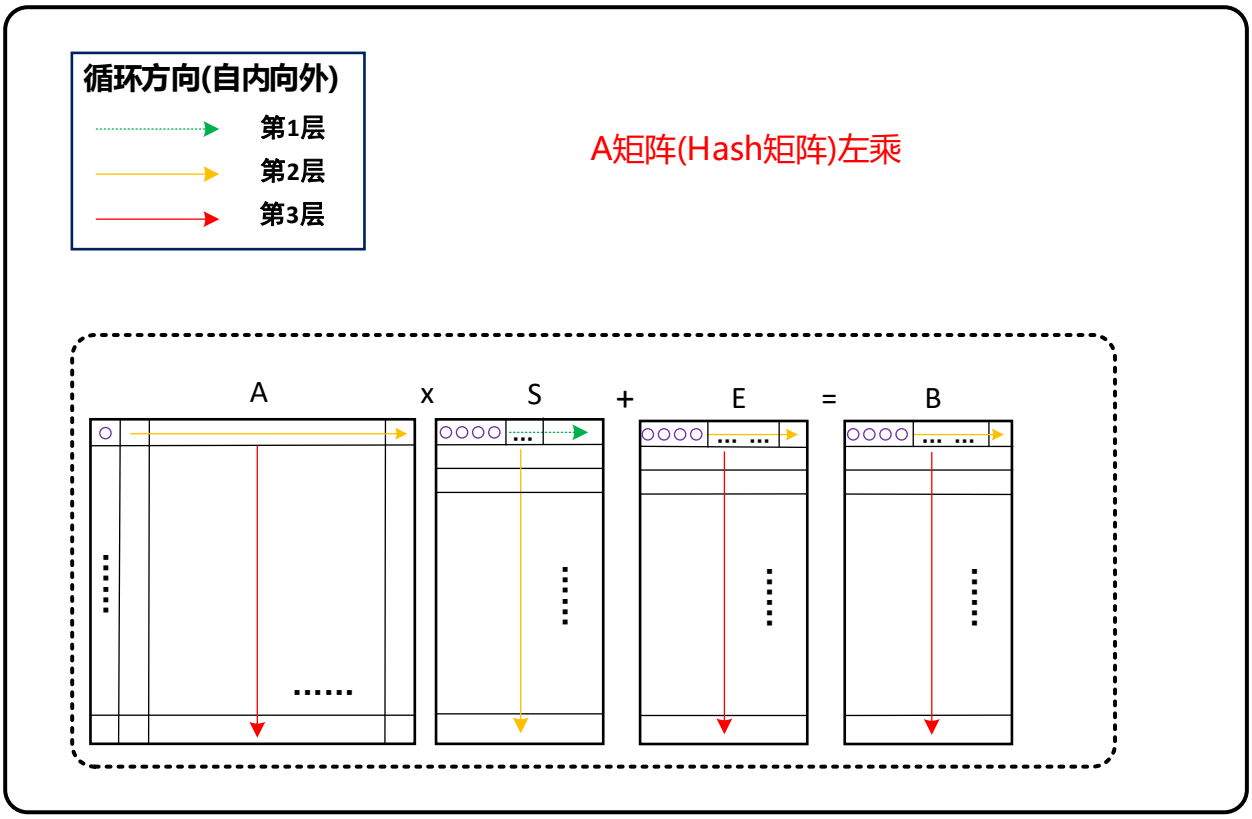


图 19　A 左乘流程设计图

如图所示，针对 A×S+E 这一 A 矩阵左乘运算，使用 4 个矩阵 PE 运算单元，每一个运算单元可进行针对 16 比特的 a×b+c（a、b、c 分别为 A、S、E 矩阵中的元素）数学运算。

A 以自上至下自左至右的遍历方向，自矩阵 A 取出第 1 行第 1 个元素，S 矩阵以自左至右方向将第 1 行元素逐个与第 1 个 A 矩阵元素进行乘法运算，E 矩阵以自左至右方向将第 1 行元素逐个与乘积进行加法运算，此时结果矩阵第一行每个元素仅是其中间结果。之后自矩阵 A 取出第 1 行第 2 个元素，S 矩阵以自左至右方向将第 2 行元素逐个与第一个 A 矩阵元素进行乘法运算，乘积逐个与对应的中间运算结果相加。重复此过程直至 A 的第一行遍历结束，此时结果矩阵第一行元素生成完成。对于 A 的每一行都采用此流程（仅在 A 本行第 1 个元素时 E 遍历一行），直至 A 矩阵遍历完毕，此时结果矩阵每一个元素生成完成。A 矩阵不存储，通过 Hash 函数按行生成 A 的元素，每次生成一个，按流程运算后生成下一个。

总结而言，对于 A 的一行，S 遍历整个矩阵一轮，对于 A 每一行的第一个元素，E 遍历对应的一行。

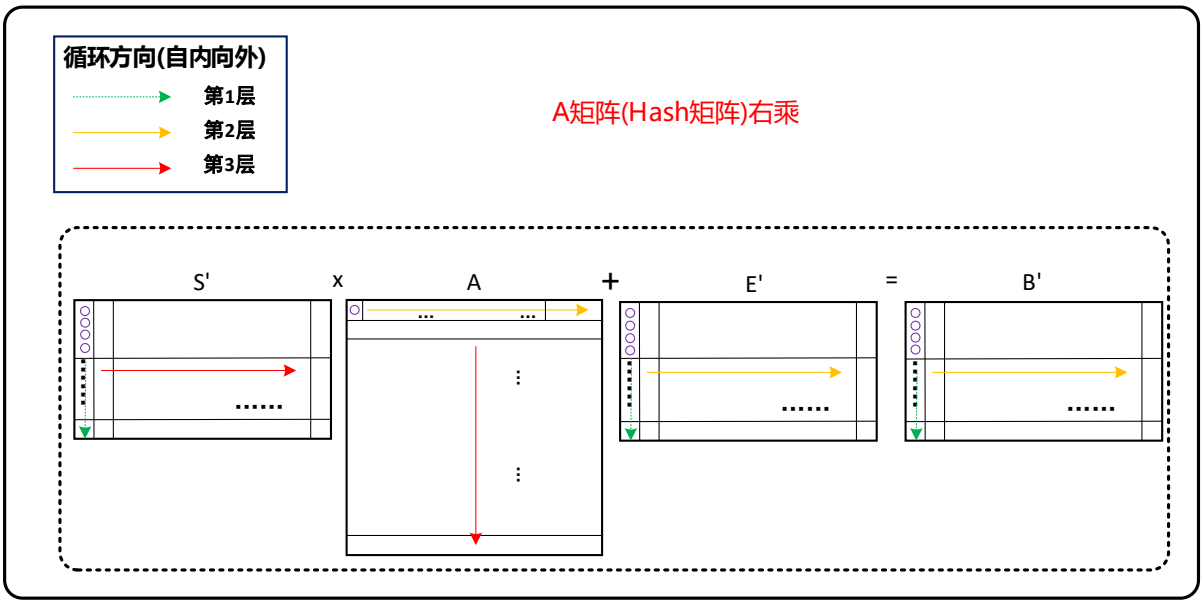


图 20　A 右乘流程设计图

如图所示，针对 S'×A+E'这一类型的 A 矩阵右乘运算，使用 4 个矩阵 PE 运算单元。为了避免按列读取 A 矩阵，本设计更换了矩阵乘法的运算顺序，每一次计算一个8×1的矩阵和$1 \times n$的矩阵之间的乘法，得到结果矩阵的暂时结果。此外，在计算过程中，由于在本设计中矩阵均为短边连续存储，因此 S'和 E'均为简单的顺序读取。

针对 A 不参与的矩阵运算，其参与乘法的矩阵均在运算前已被生成并存储，可以直接采用类似的运算流程，由于不需控制 Hash 模块，控制更加简单。由于本设计采用了的是特殊设计的 8bit×16bit 的乘法器，因此对于 A 不参与的矩阵运算，均选择 S 矩阵右乘的方式，对于 S 左乘的情况，在矩阵乘法运算结束后，会进行一个额外的转置操作。

2.4. SRAM 存储规划

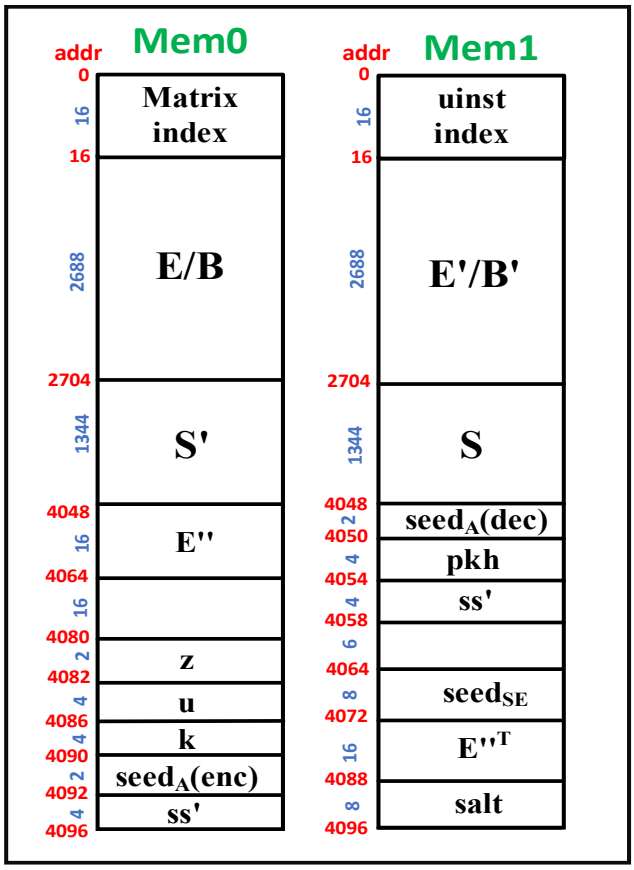


图 21　存储地址示意图

在本文中，共使用了 2 块 4096*64bit 大小的 DpRAM，用于存储数据。FrodoKEM 算法的硬件实现，必须存储的数据包括公钥（sk），私钥（sk），共享秘密（ss、ss‘），密文（c）。而对于矩阵运算过程中的中间数据存储，可进行优化设计，通过新数据覆盖旧数据的方法，达到不同中间数据公用同一块存储空间的效果，进而减小存储资源的使用。此外，在每个 RAM 的起始 16 位地址都开辟了一块空间，分别用于存储每个矩阵的首地址和指令对应的微指令的首地址。需要说明，对于 Frodo640 等级，本设计中规定矩阵元素位宽为 16 位，通过此种方法实现不同安全等级下的位宽规划统一，并且通过内部电路设计不影响最终运算结果的正确性。

多个数据复用一个 RAM 存储空间的详细说明：

1. 矩阵 E 与矩阵 B，矩阵 E'与矩阵 B'，其矩阵维度均一致。矩阵 E 参与矩阵 B 的加法运算，矩阵 E'同理。设计 E 与 B 共用一块存储资源，E'与 B'共用一块存储资源。
2. 矩阵 E''、矩阵 V、矩阵 M、矩阵 C(Dec)，四者矩阵维度一致，且分别为同一个矩阵运算过程下的待加矩阵与结果矩阵，设计四者共用一块存储资源。
3. 矩阵 E''^{T}、矩阵 V^{T}、encode(u)、矩阵 C(Enc)，四者所需存储空间一致，且同属一个矩阵运算过程中的相关数据，设计四者共用一块存储资源。

4. 比特串 SeedSE 与 Seed'SE，二者数据大小一致，均作为输入消息，哈希产生需采样的数据，设计二者共用一块存储资源。
5. k 与 k'，二者数据大小一致，在相似的运算过程中作为哈希值输出，设计二者共用一块存储资源。

2.5. 矩阵短边存储策略

初次之外，本设计对所有的矩阵采用了一种短边存储的方式，即矩阵的短边连续存储，易于读写和处理，下图展示了两种生成情况是如何进行存储的。

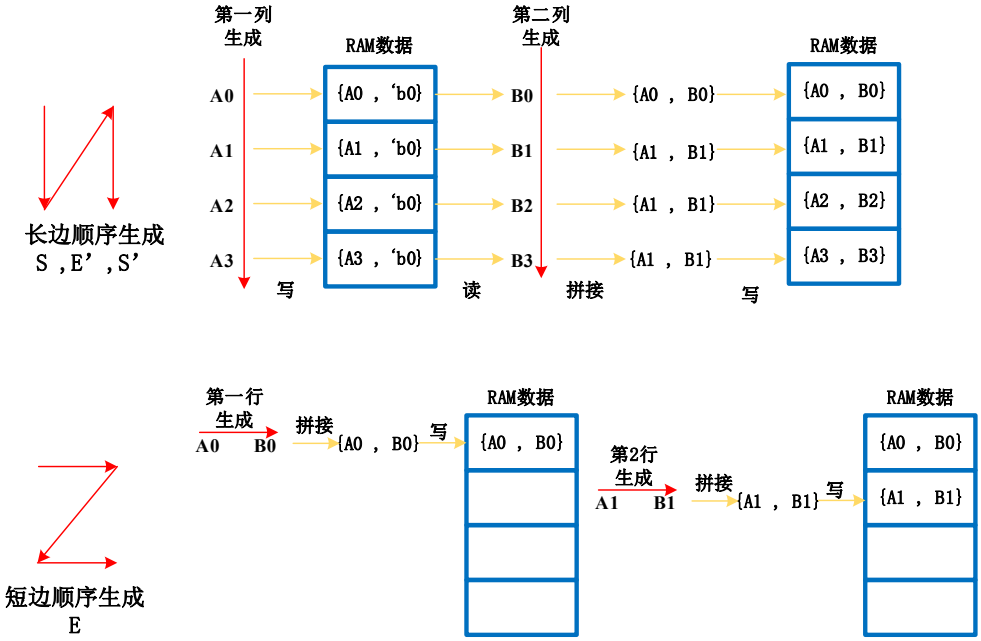


图 22　存储地址示意图

3. 子模块原理与结构设计

3.1. Hash 系数生成模块

3.1.1.接口设计

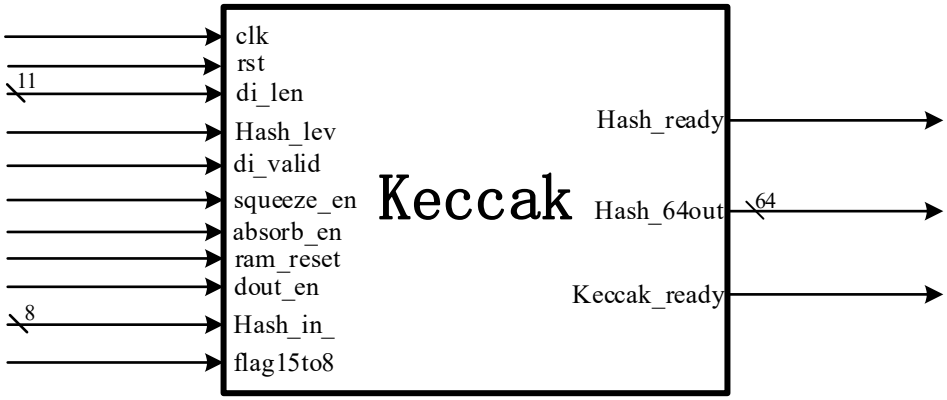


图 15　Hash 系数生成模块接口图

表 4　Hash 模块接口定义

端口名称	I/O	位宽	定义
clk	input	1	时钟
rst	input	1	复位信号
di_len	input	11	输入数据长度
Hash_lev	input	1	模式选择：SHAKE128/256
di_valid	input	1	数据输入使能
suqeeze_en	input	1	挤压使能
absorb_en	input	1	吸收使能
ram_reset	input	1	分布式 ram 复位使能
dout_en	input	1	数据输出数据
Hash_ready	output	1	总计算就绪(可进行新一次哈希)
Keccak_ready	output	1	子计算就绪(可进行下一步计算)
Hash_64out	output	64	输出数据

3.1.2.结构与功能设计

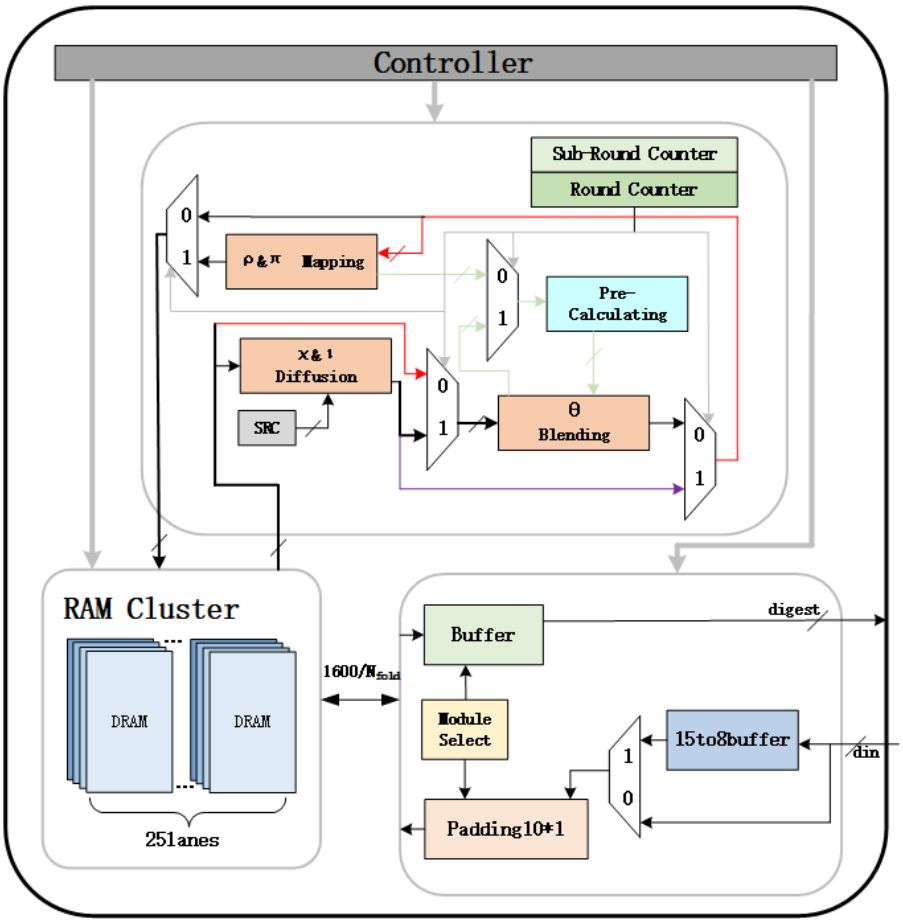


图 16　Hash 系数生成模块结构图

在 FrodoKEM 算法中，可通过两种方法进行矩阵 A 元素的生成，一种为通过 AES 函数，另一种为通过 SHAKE 函数。由于在 FrodoKEM 中 SHAKE 函数被多次使用（如 seedA，ss 的生成），本设计选择使用 SHAKE 函数。具体而言选择使用的 Hash 函数为 SHAKE128 与 SHAKE256，在 FrodoKEM 的实现过程中哈希模块会占总工程资源开销的很大一部分，针对哈希开销过大的问题，本设计使用切片技术以循环折叠的方法设计哈希模块。

哈希模块设计的核心思路为将 24 轮 keccak 运算的每 1 轮分为 8 次进行运算，即将 1600 位状态数组切片为 8 部分，每次仅处理其中一部分数据（200bit），进而降低运算所需的组合逻辑资源，同时将 Vivado 的分布式 ram 作为数据的存储载体，降低了 1600 为状态数组暂存所需的寄存器资源。

在切片设计下的会有数据依赖性问题出现，体现在 ρ 、 θ 、 π 运算中，采取调整 keccak 轮运算的顺序的方法，如图 17 所示将 ρ 与 π 步骤放在每一轮运算的最后方，解决了 ρ 与 π 的数据关联性问题。并设计了数据预处理模块(pre_calculate)在 θ 运算时为其提供相邻切片的所需数据解决了 θ 数据关联性问题。

$$r_i = \begin{cases} \theta \circ \rho \circ \pi & if\ i = 0 \\ \chi \circ \iota \circ \theta \circ \rho \circ \pi & if\ 1 \le i \le 23 \\ \chi \circ \iota & if\ i = 24 \end{cases}$$

图 17　keccak 运算子步骤顺序

如图 16 所示，Hash 模块内部逻辑主要可分为四部分 1.输入输出处理部分 2.分布式 ram 集群 3.keccak 运算核心 4.总控。下面介绍工作流程，数据在进入哈希模块后，会被写入分布式 ram 中，在数据长度不足一次吸收时在 ram 的特定位置写入 4'b1000 与 4'b0001,通过此种方法完成输入与填充过程。对于吸收与挤压所需的 Keccak 运算，如图 x 所示 ρ 与 θ 子运算通过对 ram 的取址读写来实现，其余子运算被设计为针对 200bit 而非 1600bit，通过取址与子运算模块的协同工作完成 Keccak 运算。在 Keccak 运算结束后会将数据从分布式 ram 中写入到输出 buffer 中，等待输出使能有效进而输出。

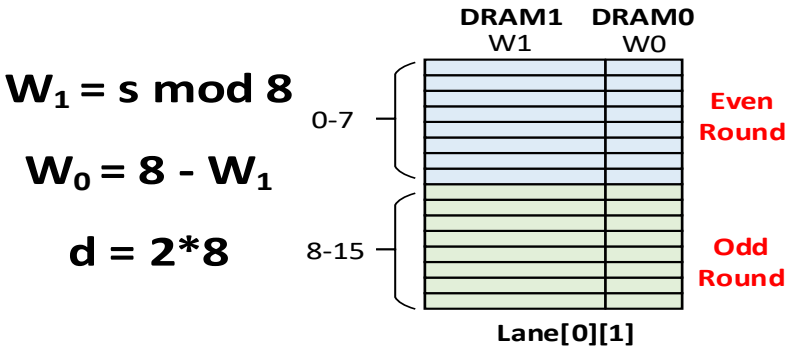


图 18　分布式 ram 存储格式

如图 18 所示，分布式 ram 以 Lane 的三维状态数组划分进行数据存储，其中 s 为 ρ 步骤中的数据实际位移量，根据 s 规划分布式 ram 的数量与宽度。数据采取交替读写的方法，在偶数轮 Keccak 从 0-7 地址读出写入进 8-15 地址，奇数轮相反，避免了数据的覆盖。

需要说明，针对 Frodo-640 等级下吸收矩阵的应用场景，由于本模块的位宽被设计 8 位，且此等级下矩阵元素位宽被设计为 16 位，因此设计 15to8buffer 用来将输入的 8 位数据重排为 15 位的有效数据，提高了模块的通用性。

3.2. 高斯采样模块

3.2.1.接口设计



图 19　高斯采样模块接口图

表 5　采样器模块接口定义

端口名称	I/O	位宽	定义
clk	input	1	时钟
rst_n	input	1	下降沿复位信号
valid_i	input	1	使能信号
level_i	input	2	安全等级选择信号
Random_i	input	16	待采样数据(1 个)
1sample_o	output	8	采样结果（1 个）
valid_o	output	1	输出有效

3.2.2.结构与功能设计

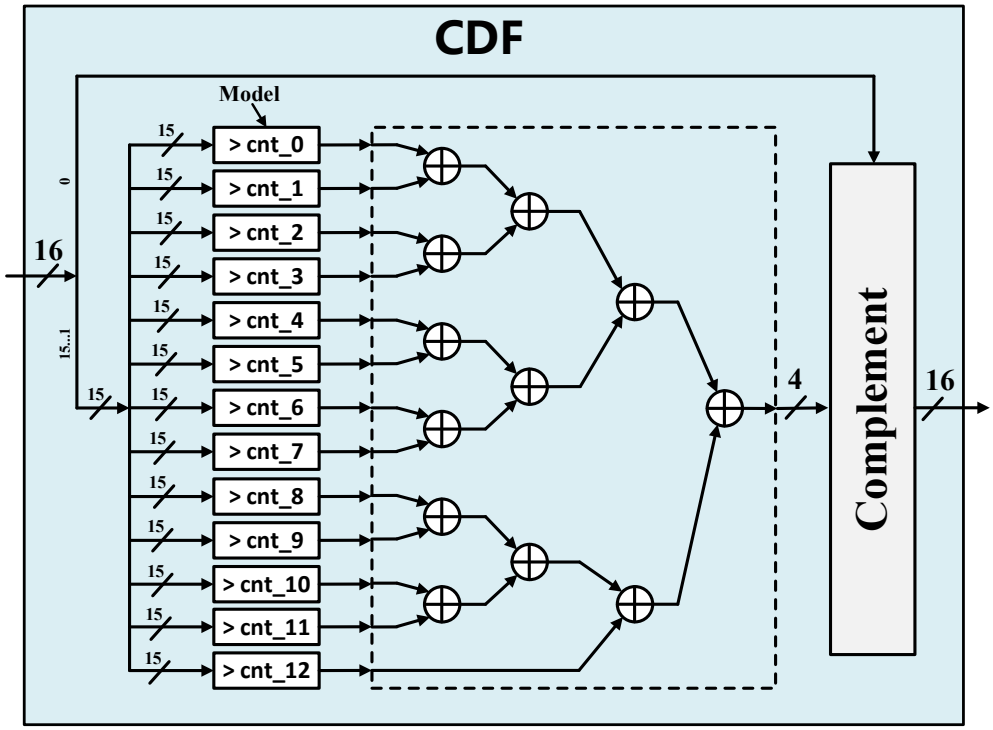


图 20 概率密度函数硬件结构

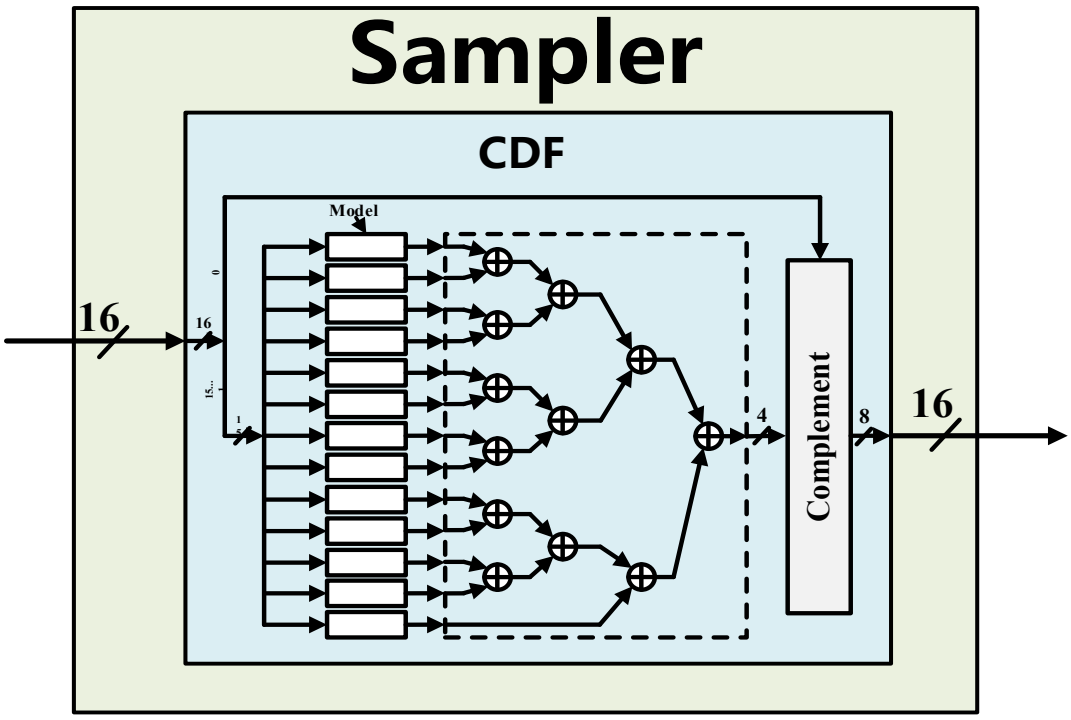


图 21 高斯采样硬件结构

哈希函数生成的部分数据需在经过高斯采样才可后组成矩阵。针对高斯采样过程，本文设计了高斯采样器的硬件结构，实现了对数据进行高斯采样后输出的功能。

如图所示，采样器仅由 1 个 CDF 模块组成，CDF 模块主要结构由数个数值比较器，加法树，符号处理模块组成。采用将误差分布表中数据提前储存在硬件单元中的方法，

降低控制难度，有利于减少控制所需的硬件资源。

数据在输入采样器后，最低有效位数据会输入符号处理模块，其余有效位组成的数值在安全等级信号的控制下，查找对应的误差分布表，逐个与表中数据比较大小，小于此数值的表中数据个数即为采样结果的绝对值。绝对值进入符号处理模块后，根据最低有效位得数值进行正负符号的添加，最终得到采样数据输出。

3.3. 矩阵运算 PE 阵列

3.3.1.接口设计



图 22 矩阵运算模块接口图

表 6　矩阵运算 PE 阵列接口定义

端口名称	I/O	位宽	定义
clk	input	1	时钟
rst_n	input	1	复位
en_i	input	1	使能
A_i	input	16	左乘矩阵 1 个元素
B_i	input	64	右乘矩阵 4 个元素(1 个元素 16bit)
C_i	input	64	待加矩阵 4 个元素(1 个元素 16bit)
dout_o	output	64	输出 4 个结果矩阵元素(1 个元素 16bit)
valid_o	output	1	输出有效

3.3.2.结构与功能设计

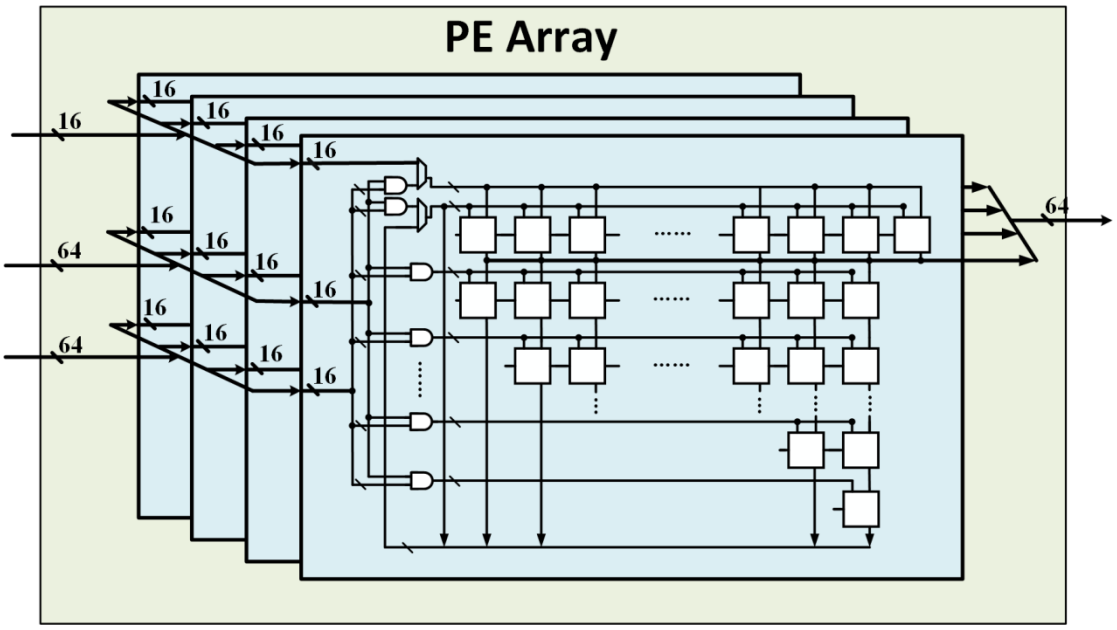


图 23　PE 运算单元阵列

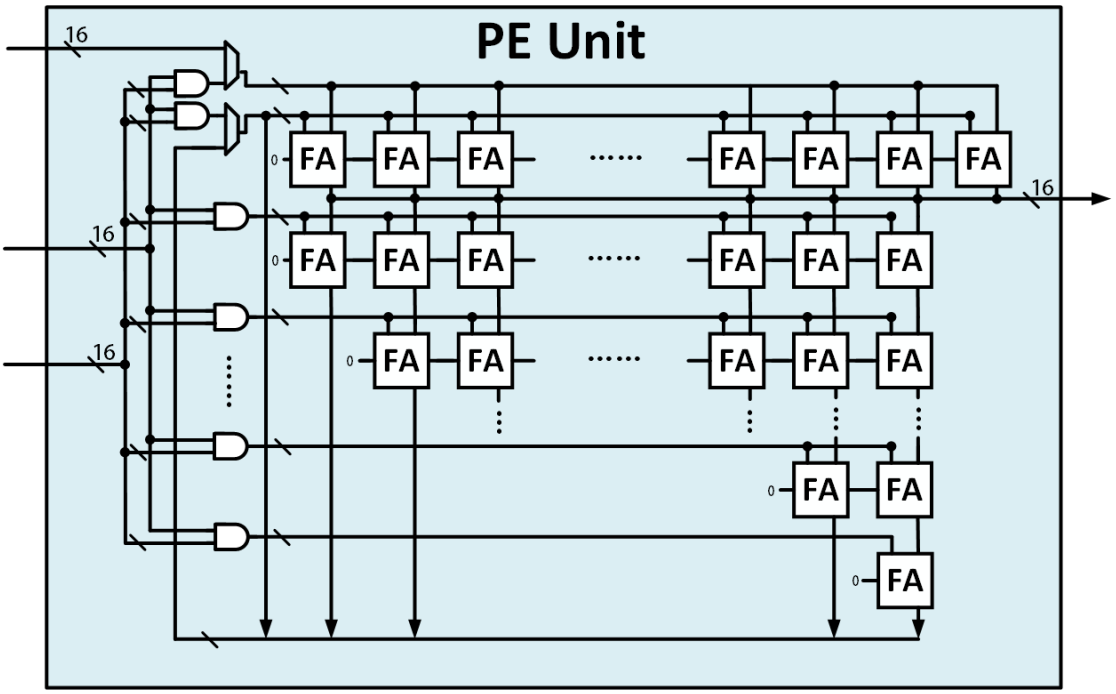


图 24　全加器阵列（乘加器结构）

在 FrodoKEM 算法运行过程中，需要对大维度矩阵进行运算，包括矩阵乘法与矩阵加法，需要设计专用的矩阵运算 PE 阵列，本设计采用根据运算数据与运算流程的特点，通过手动设计全加器（Full Adder，FA）阵列的方式，实现乘法器加法器公用结构，有利于减少硬件资源的使用。

在 FrodoKE 算法中，矩阵运算分为两个内容，矩阵数据模乘与矩阵数据模加，即 A

× B + C(A、B、C 为参与运算的矩阵)。矩阵运算 PE 阵列由四个 PE 运算单元组成，每一个运算单元可均以实现对输入其的三个数据进行 （a × b + c） mod 65535 的数学运算，整个 PE 阵列可以并行 4 个 PE 运算单元，在控制模块的控制下，PE 阵列可以完成对于 FrodoKEM 中矩阵的运算。

如图所示，PE 阵列由 4 个 PE 运算单元组成，PE 单元的本质是一个由全加器构成的阵列，根据 FrodoKEM 算法中的数据运算特点，在传统全加器阵列乘法器上进行优化设计。算法中的矩阵数据在运算结束后需通过 Pack 操作，舍弃高有效位数据，仅保留 16 比特。在本设计中，为降低乘法器加法器资源开销，将其中的乘法运算设计为模乘与模加运算，对应在全加器阵列中删除运算高有效位数据的全加器，构成适用于算法的模乘器，并使用模乘器中的部分全加器进行模加运算，实现了模乘模加运算公用一套硬件结构，完成在此运算单元上的低资源开销设计。

数据在输入 PE 运算单元后，首先两个乘数每一位经过与运算，得到多个部分和，分别被输入到整个全加器阵列中，实现模乘运算。之后得到的乘积与加数会再次被输入加法器阵列，使用阵列中的部分全加器阵列实现模加运算，最终输出运算结果。

3.4. Encode 模块

3.4.1.接口设计

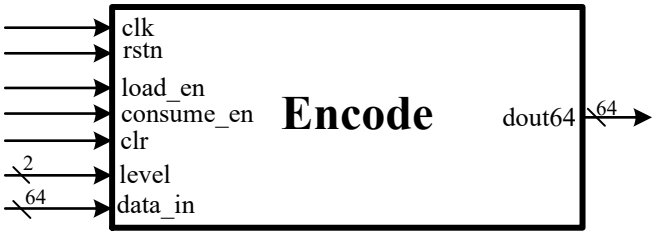


图 25　Encode 模块接口图

表 7　Encode 模块接口定义

端口名称	I/O	位宽	定义
clk	input	1	时钟
rstn	input	1	复位
load_en	input	1	数据载入使能
consume_en	input	1	数据输出使能
clr	input	1	内部计数器清零信号
level	input	2	安全等级指示信号
data_in	output	64	输入数据
data_out	output	64	输出数据

3.4.2.结构与功能设计

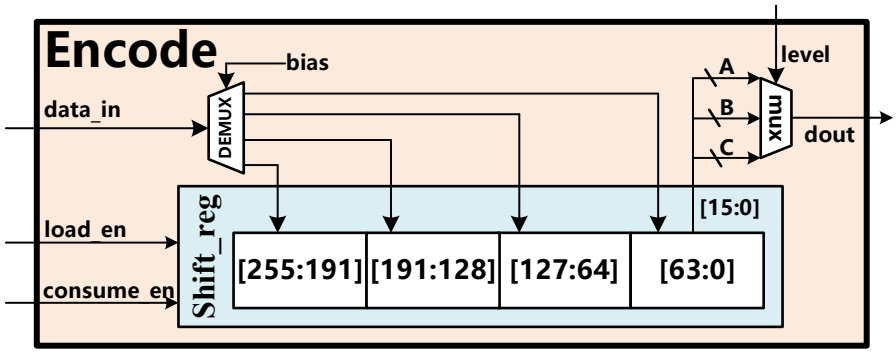


图 26　Encode 模块硬件结构图

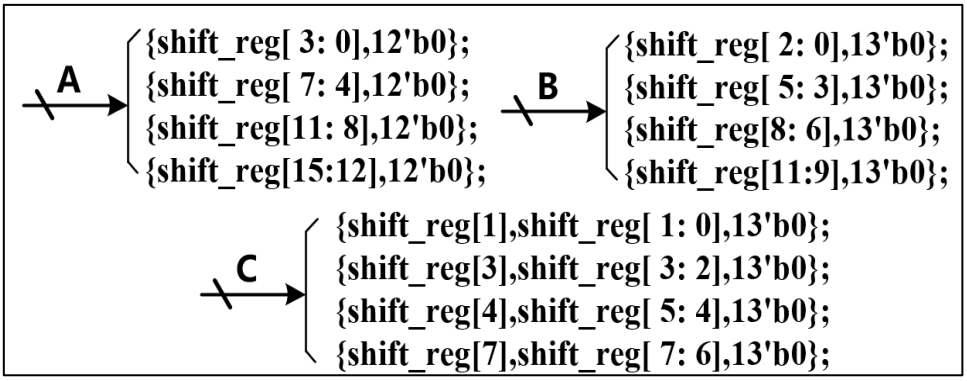


图 27　A、B、C 三路数据具体组成示意图

针对将位串编码为矩阵的运算，设计了 Encode 模块，其可进行三种安全等级下的 Encode 运算。

如图所示，Encode 模块功能依靠内部一个 256 位的多功能移位寄存器(shift_reg)实现，其在输入使能有效或输出使能有效的情况，可分别进行向左与向右的移位，前者在内部计数器 bias 信号的控制下，控制输入数据装载入移位寄存器的不同部分，后者配合数据输出。从移位寄存器的低位取出数据，之后分为三路分别进行数据组合进而获得 A、B、C 三路数据，根据安全等级信号的数值选择输出哪一路数据，需要说明，对于 Frodo640 等级(C 信号),本设计中规定结果数据(矩阵元素)位宽为 16 位，与上文存储规划保持一致。通过以上方法设计电路实现将位串编码为矩阵的运算。

3.5. Decode 模块

3.5.1.接口设计

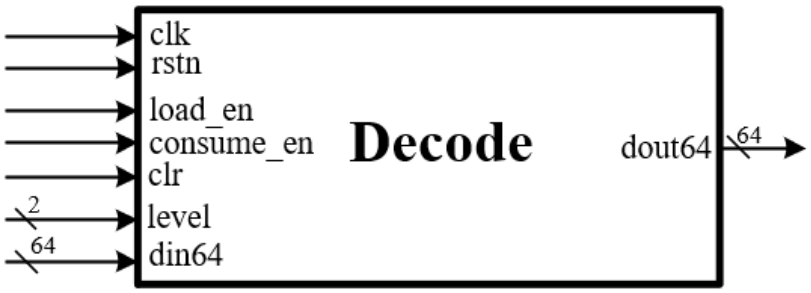


图 28　Encode 模块接口图

端口名称	I/O	位宽	定义
clk	input	1	时钟
rstn	input	1	复位
load_en	input	1	数据载入使能
consume_en	input	1	数据输出使能
clr	input	1	内部计数器清零信号
level	input	2	安全等级指示信号
din64	output	64	输入数据
dout64	output	64	输出数据

3.5.2.结构与功能设计

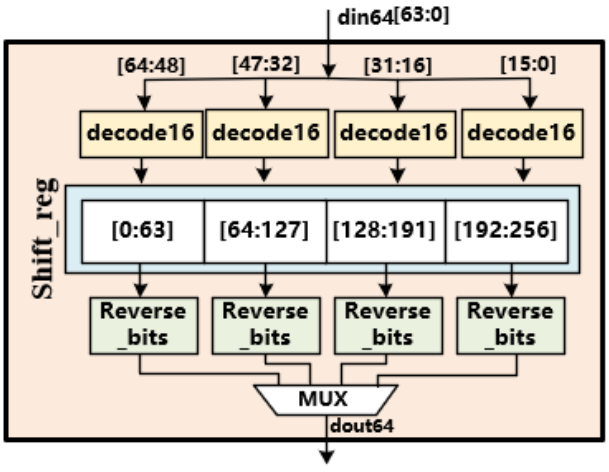


图 29　Encode 模块硬件结构图

针对将矩阵处理为位串的运算，设计了 Decode 模块，其可进行三种安全等级下的 Decode 运算。

如图所示，Decode 模块由 decode16、多功能移位寄存器(Shift_reg)、Rverse_bits 组成。数据在输入后被分为 4 部分，分别进入 decode16 进行运算，之后被输入移位寄存器中，寄存器中数据在经过 Rverse_bits 即将 64 位数据倒序反转后选择输出。通过以上方法设计电路实现将矩阵处理为位串的运算。

3.6. 控制模块

3.6.1.接口设计

表 7　控制模块接口定义

端口名称	I/O	位宽	定义
clk	input	1	时钟
reset	input	1	复位信号
absorb_i	output	1	Hash 吸收使能
squeeze_i	output	1	Hash 挤压使能
mode_sel_i	output	1	模式选择：SHAKE128/256
bitlen_i	output	11	输入长度指示信号
64to60_i	output	1	位宽转换指示信号
hash_ready_i	output	1	输入使能
din_i	output	64	输入数据
flag_i	output	1	输入移位寄存器清零信号
squeeze_cnt_i	output	2	挤压轮数指示信号
out_sel_i	output	2	输出模式指示信号
stop_o	output	1	输出暂停信号
valid_i	output	1	采样使能信号
level_i	output	2	安全等级选择信号
en_i	output	1	PE 阵列使能

A_i	output	16	左乘矩阵 1 个元素
B_i	output	64	右乘矩阵 4 个元素(1 个元素 16bit)
C_i	output	64	待加矩阵 4 个元素(1 个元素 16bit)
dout_o	input	64	输出 4 个结果矩阵元素(1 个元素 16bit)
ram0_addra	output	13	RAM0 端口 a 地址
ram0_addrb	output	13	RAM0 端口 b 地址
ram0_dina	input	64	RAM0 端口 a 输入数据
ram0_dinb	input	64	RAM0 端口 b 输入数据
ram0_douta	output	64	RAM0 端口 a 输出数据
ram0_doutb	output	64	RAM0 端口 b 输出数据
ram0_ena	output	1	RAM0 端口 a 写使能
ram0_enb	output	1	RAM0 端口 a 写使能
ram1_addra	output	13	RAM1 端口 a 地址
ram1_addrb	output	13	RAM1 端口 b 地址
ram1_dina	input	64	RAM1 端口 a 输入数据
ram1_dinb	input	64	RAM1 端口 b 输入数据
ram1_douta	output	64	RAM1 端口 a 输出数据
ram1_doutb	output	64	RAM1 端口 b 输出数据
ram1_ena	output	1	RAM1 端口 a 写使能
ram1_enb	output	1	RAM1 端口 a 写使能
ram2_addra	output	13	RAM2 端口 a 地址
ram2_addrb	output	13	RAM2 端口 b 地址
ram2_dina	input	64	RAM2 端口 a 输入数据

ram2_dinb	input	64	RAM2 端口 b 输入数据
ram2_douta	output	64	RAM2 端口 a 输出数据
ram2_doutb	output	64	RAM2 端口 b 输出数据
ram2_ena	output	1	RAM2 端口 a 写使能
ram2_enb	output	1	RAM2 端口 a 写使能

3.6.2.功能设计

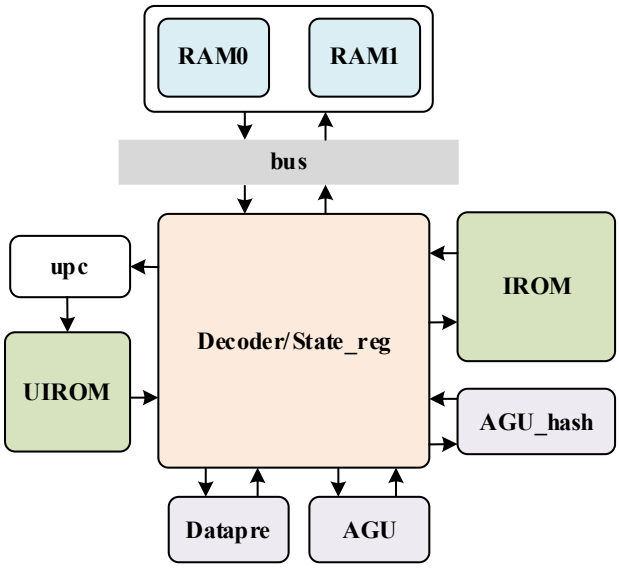
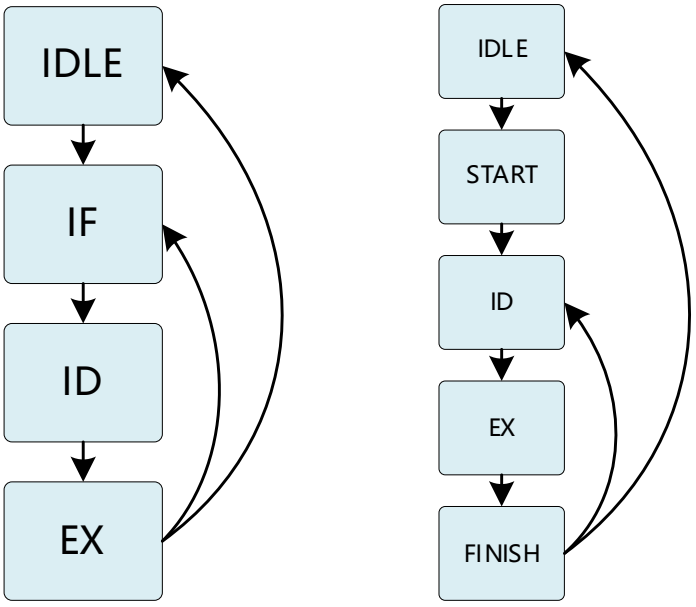


图 控制模块架构图

本设计使用了一种算法-指令-微指令的二级调度机制，根据执行的算法，选择对应的指令段，执行每个指令时，再选择对应的微指令段，实现以较少的存储资源换取控制模块的逻辑资源。



如图所示，控制模块仅有两个简单的状态机。左图为指令的状态机，待机状态时，当系统收到start 信号开始根据选择的算法和安全等级进行取值和简单译码，之后便进入该指令的执行阶段。右图为微指令的状态机，指令开始执行时，微指令状态开始运行。在 ID 阶段，根据指令的字段，获取相应的控制寄存器信息，并从 ram 中读取相关操作矩阵的首地址和微指令段的首地址。在 EX 阶段，控制模块不断从 UIROM 中读取微指令，微指令中的控制信号直接输出至外部对应的模块。

控制模块中的模块功能如下。Datapre 模块进行简单的数据预处理，如截取、拼接；AGU 模块产生矩阵乘法时的地址；AGU_hash 生成 hash 相关指令时需要的地址；bus 为控制模块与存储模块访存的总线；IROM 存储各个算法对应的指令段；UIROM 存储各个指令对应的微指令段。